DHVANI — Built-in DAC Build

v3 · STM32U575ZI-Q internal 12-bit DAC · replaces the DAC0800 / breadboard plan

Work through this in order. Each step ends at a gate — don't start the next step until the gate passes. The DAC0800, the −9 V battery and the TL074 breadboard are no longer needed for the core system.

What changed from v2

Item	v2 (breadboard)	v3 (this PDF)
DAC	External DAC0800, 8-bit	**U575 internal DAC1**, 12-bit, output on PA4
Sample clock	TIM1 → DMA → GPIO port	**TIM6 TRGO** triggers the DAC; DMA feeds it
DMA request	TIM1_UP, byte to GPIOE->ODR	**DAC1_CH1**, half-word to the DAC
Supplies	+5 V and −9 V battery	**USB only**
Analog chain	DAC0800 + TL074 I→V, filter, buffer	**Passive RC filter** on PA4 (TL074 optional)
Sample values	8-bit, centre 128	**12-bit, centre 2048**, amplitude 1800
Output range	0 – 2.9 V	~0.2 – 3.1 V (DAC output buffer on)

The architecture

```
Turbidity / TDS ──► ADC1 ──► firmware picks the chirp band
Firmware builds a Hann-windowed chirp ──► uint16_t buffer in RAM
TIM6 update (1 MHz) ──► DAC1 trigger ──► GPDMA1 feeds the next sample
DAC1 OUT1 (PA4) ──► RC filter ──► scope     PA4 ──► 1 kΩ ──► buzzer
CPU sleeps (__WFI) while each 20 ms ping plays
```

Where PA4 is on the board

Location	Notes
Zio **CN7 pin 9** (D20) or **CN7 pin 17** (D24)	Female sockets — try these first. They only carry PA4 if the board's solder bridge for that pin is closed.
Morpho **CN11 pin 32**	Always wired to PA4 — use this if the Zio pins show nothing.
GND	CN8 pin 11, or any GND on the headers

STEP 1 · CUBEMX (about 10 minutes)

Open the existing `sih.ioc`. The 160 MHz clock, ICACHE, LEDs and BSP are already correct. PE0–PE11 and TIM1 can stay — they're harmless.

Analog → DAC1

Setting	Value
OUT1 mode	Connected to external pin only (PA4)
Mode	Normal
Output Buffer	Enable
Trigger	Timer 6 Trigger Out event

Timers → TIM6

Setting	Value
Activated	✓
Prescaler	0
Counter Period	**159** → 160 MHz ÷ 160 = 1 MSps
Trigger Event Selection (TRGO)	Update Event

System Core → GPDMA1 → Channel 0

Setting	Value
Mode	Standard Request Mode
Request	DAC1_CH1
Direction	Memory to Peripheral
Source / Destination data width	Half Word / Half Word
Source increment	Enabled
Destination increment	Disabled
DMA handle in IP structure (if shown)	DAC1 → DMA_Handle1
NVIC	GPDMA1 Channel 0 global interrupt — Enabled

Analog → ADC1 (sensors)

Setting	Value
Channels	IN8 = PA3 (Zio A0, CN9 pin 1) · IN4 = PC3 (Zio A2, CN9 pin 5)
Resolution	12-bit (the U5 defaults to 14 — change it)
Scan Conversion Mode	Enabled · Number of Conversion 2 · Rank 1 = IN8 · Rank 2 = IN4
End Of Conversion Selection	End of single conversion
Sampling time	Long (68 cycles or more) for both ranks
Clock	If the Clock tab turns red on the ADC, set the ADC clock mux to HSI

Generate code.

STEP 2 · DC TEST (Gate 2)

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start(&htim6);
HAL_DAC_Start(&hdac1, DAC_CHANNEL_1);
static const uint16_t test[3] = { 0, 2048, 4095 };
/* USER CODE END 2 */

  /* USER CODE BEGIN 3 */
  for (int i = 0; i < 3; i++)
  {
    HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_1, DAC_ALIGN_12B_R, test[i]);
    BSP_LED_Toggle(LED_GREEN);
    HAL_Delay(3000);
  }
}
/* USER CODE END 3 */
```

Keep the closing } before `USER CODE END 3` — it's the one CubeMX generated for `while (1)`. Don't add another.

GATE 2 — PA4 against GND

Steps every 3 seconds: **~0.2 V → ~1.65 V → ~3.1 V**. (With the buffer on, the DAC can't quite reach 0 V or 3.3 V.)

The middle reading should be about halfway between the other two.

Symptom	Likely cause
Nothing on CN7 pin 9 or 17	Solder bridge open — measure CN11 pin 32 instead
Always 0 V everywhere	Code not flashed (check LD1 flips every 3 s), or DAC1 OUT1 not enabled in CubeMX
Stuck at one value	TIM6 not started, or DAC trigger not set to Timer 6
Build error: htim6 / hdac1 undeclared	TIM6 or DAC1 not enabled before generating code

STEP 3 · DMA PINGS — FINAL FIRMWARE (Gate 3)

Replace the user code sections with these. This is the complete final firmware; set `AUDIBLE 1` and ignore the sensors until step 5.

```
/* USER CODE BEGIN Includes */
#include <math.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
#define N 20000                    // 20 ms ping at 1 MSps
#define AUDIBLE 1                  // 1 = buzzer band, 0 = ultrasonic band
static uint16_t wave[N];
volatile uint8_t ping_done = 1;
uint32_t turbidity = 0, tds = 0;
/* USER CODE END PV */
```

```
/* USER CODE BEGIN 0 */
static float sample_rate(void)
{
  return 160000000.0f / (float)(TIM6->ARR + 1);
}

static void build_tone(float f)                 // Hann-windowed tone burst
{
  float fs = sample_rate();
  for (int i = 0; i < N; i++)
  {
    float w = 0.5f * (1.0f - cosf(2.0f * 3.14159265f * i / (N - 1)));
    wave[i] = (uint16_t)(2048.0f + 1800.0f * w * sinf(2.0f * 3.14159265f * f * i / fs));
  }
}

static void build_chirp(float f0, float f1)     // Hann-windowed linear chirp
{
  float fs = sample_rate();
  float T = N / fs, k = (f1 - f0) / T;
  for (int i = 0; i < N; i++)
  {
    float t  = i / fs;
    float ph = 2.0f * 3.14159265f * (f0 * t + 0.5f * k * t * t);
    float w  = 0.5f * (1.0f - cosf(2.0f * 3.14159265f * i / (N - 1)));
    wave[i]  = (uint16_t)(2048.0f + 1800.0f * w * sinf(ph));
  }
}

static void fire_ping(void)
{
  ping_done = 0;
  HAL_DAC_Start_DMA(&hdac1, DAC_CHANNEL_1, (uint32_t *)wave, N, DAC_ALIGN_12B_R);
}

static void read_sensors(void)
{
  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, 10);  turbidity = HAL_ADC_GetValue(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, 10);  tds       = HAL_ADC_GetValue(&hadc1);
  HAL_ADC_Stop(&hadc1);
}
/* USER CODE END 0 */
```

```
  /* USER CODE BEGIN 2 */
  HAL_ADCEx_Calibration_Start(&hadc1, ADC_CALIB_OFFSET, ADC_SINGLE_ENDED);
  HAL_TIM_Base_Start(&htim6);                  // 1 MSps trigger
  /* USER CODE END 2 */

    /* USER CODE BEGIN 3 */
    static uint32_t avg = 0;
    read_sensors();
    avg = (avg * 3 + turbidity) / 4;            // 0..4095

#if AUDIBLE
    float f0 = 2000.0f,   f_hi = 5000.0f   + avg * 3.0f;    // up to ~17 kHz
#else
    float f0 = 100000.0f, f_hi = 120000.0f + avg * 32.0f;   // up to ~250 kHz
#endif
    float f_max = 0.25f * sample_rate();
    if (f_hi > f_max) f_hi = f_max;

    build_chirp(f0, f_hi);
    fire_ping();
    BSP_LED_Toggle(LED_GREEN);
    while (!ping_done) { __WFI(); }             // CPU sleeps while DMA plays the ping
    HAL_DAC_Stop_DMA(&hdac1, DAC_CHANNEL_1);
    HAL_Delay(200);
  }
  /* USER CODE END 3 */

/* USER CODE BEGIN 4 */
void HAL_DAC_ConvCpltCallbackCh1(DAC_HandleTypeDef *hdac)
{
  ping_done = 1;
}
/* USER CODE END 4 */
```

Test: buzzer + through 1 kΩ to PA4, buzzer − to GND. With no sensor connected yet the chirp band stays at its lowest setting.

GATE 3 — hour 15

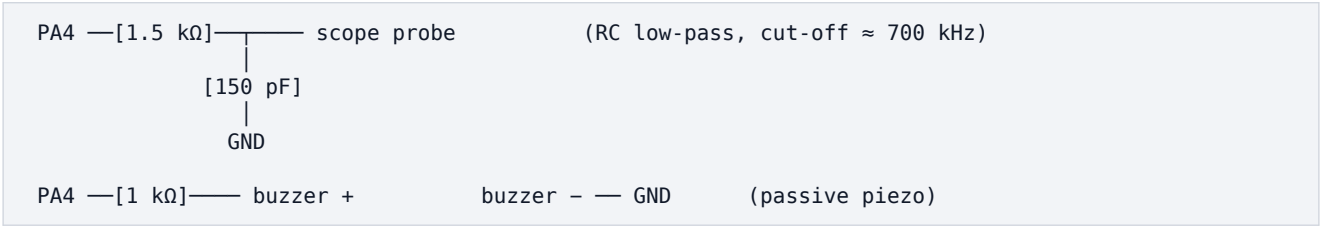
A short rising chirp every ~220 ms on the buzzer; LD1 flips each ping. On a scope: a 20 ms burst that swells and fades.

Pings only 10 ms long? Your HAL version wants the length in bytes — change `N` to `N * 2` in `fire_ping()`.

Fallback if DMA will not cooperate: a TIM6 interrupt calling `HAL_DAC_SetValue` for each sample. Lower ceiling, but it works — state the limitation honestly.

Symptom	Likely cause
No sound, LD1 not flipping	Code stuck waiting for ping_done — GPDMA1 Channel 0 interrupt not enabled in NVIC
Build error: handle / hadc1 undeclared	GPDMA1 or ADC1 not enabled before generating code
One ping, then silence	Missing HAL_DAC_Stop_DMA after the wait, or DMA left in circular mode
Frequencies all wrong	TIM6 Counter Period changed without regenerating — sample_rate() reads TIM6, check its ARR
Flat top and bottom	Reduce 1800.0f to 1600.0f in the waveform functions

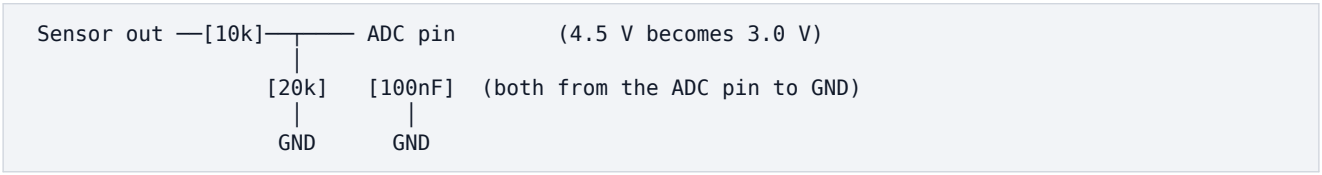
STEP 4 · OUTPUT WIRING



The DAC's own output buffer drives both loads, so no op-amp or negative supply is needed. The TL074 can be added later as a buffer if you want a lower-impedance output, but it isn't required for the demo.

STEP 5 · SENSORS AND ADAPTATION

Sensor	Power	Signal to	Note
Turbidity	5V (CN8 pin 9) + GND	PA3 · CN9 pin 1	10k/20k divider if its output exceeds 3.3 V
TDS	5V (CN8 pin 9) + GND	PC3 · CN9 pin 5	Usually below 3.3 V — measure first



Verify the direction: move the probe between clear and muddy water. The pitch should **drop** in muddy water (muddy water absorbs high frequencies more, so the transmitter moves to a lower band). If it rises, replace `avg` with `(4095 - avg)` in the `f_hi` line.

STEP 6 · CAPTURE

	Capture
☐	Demo video with `AUDIBLE 1`: probe moving between cups, chirp audible
☐	Scope photo, clear water, `AUDIBLE 0`
☐	Scope photo, muddy water, same scope settings
☐	Photo of the whole setup from above
☐	Numbers written down: 12-bit, 1 MSps, band reached, CPU asleep during pings
☐	Before filming: press every wire in, two full rehearsals

Key numbers

Parameter	Value
Resolution	12-bit (4096 levels) → ~74 dB theoretical floor
Sample rate	1 MSps (TIM6: Prescaler 0, Period 159)
Audible band	2–17 kHz chirp (AUDIBLE 1)
Ultrasonic band	100–250 kHz chirp (AUDIBLE 0), capped at fs/4
Ping	20 000 samples = 20 ms, then 200 ms quiet
Window	Hann — first sidelobe about −31 dB
Output	~0.2–3.1 V on PA4, DAC output buffer on

For the slides: the internal DAC removed the external DAC, its reference and the negative supply — fewer parts, fewer failure points, better resolution. The architecture is unchanged: a hardware timer paces DMA, DMA streams the samples, and the CPU sleeps during every ping.

The three gates

Gate	Must have	If not
1	LED blinking, 160 MHz clock	Clock problem — ask for help
2	PA4 steps 0.2 / 1.65 / 3.1 V	Check the PA4 pin location and DAC1 / TIM6 settings
3	DMA pings, CPU sleeping	Fall back to a TIM6 interrupt